Wstęp do zadań:

Celem dzisiejszych zajęć jest poznanie konstruktora z domyślnym argumentem oraz delegowanie konstruktora. Ponadto zobaczymy program gdzie jest zaszyta logika działania programu w klasie. Jedno zadanie jest wykonanie z zastosowaniem szablonu.

Wykonaj wszystkie zadania.

Ćwiczenia:

Zadanie 1. Napisz program który liczy cenę brutto. Podana jest klasa „koszt” którą należy uzupełnić o brakujące elementy. Klasa ma mieć tylko jeden konstruktor z dwoma argumentami (zmienne cena i podatek), przy czym zmienna podatek ma mieć domyślny argument 23. Następnie musimy mieć destruktor który w tym wypadku jest pusty oraz metodę „brutto” która nic nie pobiera a zwraca typ double.

Klasa: koszt

```
class koszt
{
private:
        double cena;
        int podatek;
public:
        //uzupelnic
};
```

Gdy już mamy uzupełnioną klasę spróbujmy sprawdzić działanie jej w funkcji main:

```
int main(int argc, char** argv) {

        double cen;
        int pod;

        //wywołanie konstruktora
        cout<<"Podaj cene towaru: ";
        cin>>cen;
        cout<<"Podaj stawke podatku: ";
        cin>>pod;
        koszt towar(cen, pod);
        cout<<"Cena brutto: "<<towar.brutto()<<endl;

        //wywołanie konstruktora z domyslnym argumentem
        cout<<"Podaj cene towaru: ";
        cin>>cen;
        koszt ksiazka(cen);
        cout<<"Cena brutto: "<<ksiazka.brutto()<<endl;

        return 0;
}
```

Przetestuj program czy poprawnie działa.

Zadanie 2. Jest napisany program do wyliczania mocy. Mechanizm wyliczania jest zawarty w klasie „moc”. Mamy tam 4 konstruktory. Uzupełnij brakujące metody o ciało. Klasa wygląda w ten sposób:

```
class moc
{
private:
        int volt;
        double amper;
        double wat;
        void wylicz(void) {wat=volt*amper;}

public:
        moc(): volt(0), amper(0.0), wat(0.0) {}                    //konstruktor 1
        moc(int a): volt(a), amper(0.0) {wylicz();}                //konstruktor 2
        moc(double a): volt(230), amper(a) {wylicz();}             //konstruktor 3
        moc(int a, double b): volt(a), amper(b) {wylicz();}        //konstruktor 4
        ~moc(void) {}

        void set_volt(int a) {volt=a;}
        void set_amper(int a);
        double get_wat(void);
};
```

Funkcja main działającego program wygląda w ten sposób:

```
int main(int argc, char** argv) {

        int v;
        double a;
        cout<<"Moc silnika"<<endl;
        cout<<"Podaj napiecie: ";
        cin>>v;
        cout<<"Podaj prad: ";
        cin>>a;
        moc silnik(v,a);
        cout<<"Moc silnika to: "<<silnik.get_wat()<<"W"<<endl;

        cout<<endl<<"Moc zarowki"<<endl;
        cout<<"Podaj napiecie: ";
        cin>>v;
        cout<<"Podaj prad: ";
        cin>>a;
        moc zarowka(a);
        cout<<"Moc zarowki to: "<<zarowka.get_wat()<<"W"<<endl;

        cout<<endl<<"Moc komputera"<<endl;
        cout<<"Podaj napiecie: ";
        cin>>v;
        cout<<"Podaj prad: ";
        cin>>a;
        moc komp(v);
        cout<<"Moc komputera to: "<<komp.get_wat()<<"W"<<endl;
```

```
        cout<<endl<<"Moc TV"<<endl;
        cout<<"Podaj napiecie: ";
        cin>>v;
        cout<<"Podaj prad: ";
        cin>>a;
        moc tv;
        cout<<"Moc TV to: "<<tv.get_wat()<<"W"<<endl;

        return 0;
}
```

Zastanów się i uzupełnij tabelkę który konstruktor jest wywoływany

Obiekt:	Konstruktor:
silnik	
zarowka	
komp	
tv	

Zadanie 3. Program ma za zadanie wyliczenie kosztu zużycia paliwa. W metodzie „wylicz” należy użyć takiego wzoru: „(zuzycie/100)*cena”. Poniżej jest podana klasa koszt. Uzupełnij jej metody aby program działał. W tej klasie widzimy dwa konstruktory. Pierwszy jest domyślny a drugi jest parametrowy. Gdy popatrzymy na pierwszy konstruktor widzimy po dwukropku wywołanie konstruktora parametrowego z dwoma parametrami (w tym wypadku 0 i 0). Taki zapis nazwany jest **„delegowaniem konstruktora”**. Gdy w funkcji main użyjemy konstruktora bez parametrowego program wywoła konstruktor parametrowy z wpisanymi wartościami (w tym wypadku 0 i 0).

Kod klasy „koszt”:

```
class koszt
{
private:
        double zuzycie;
        double cena;

public:
                //konstruktor 1 domyslny
        koszt(): koszt(0.0,0.0) {}
                //konstruktor 2 parametrowy
        koszt(double x, double y): zuzycie(x), cena(y) {}
        ~koszt(void){}

        void set_zuzycie(double x);
        void set_cena(double x);
        double get_zuzycie(void);
        double get_cena(void);

        double wylicz(void);
};
```

Uzupełnij metody o ciało.

Funkcja main:

```
int main(int argc, char** argv) {

        double z;
        double c;
        cout<<"Policzmy koszt jednego kilometra dla Opel: "<<endl;
        cout<<"Podaj zuzycie paliwa: ";
        cin>>z;
        cout<<"Podaj cene paliwa: ";
        cin>>c;
        koszt opel;             //konstruktor domyslny
        cout<<"Koszt: "<<opel.wylicz()<<" zl. za km"<<endl;                    //(1)
        opel.set_zuzycie(z);
        opel.set_cena(c);
        cout<<"jeszcze raz koszt: "<<opel.wylicz()<<" zl. za km"<<endl<<endl;  //(2)

        cout<<"Policzmy koszt jednego kilometra dla VW: "<<endl;
        cout<<"Podaj zuzycie paliwa: ";
        cin>>z;
        cout<<"Podaj cene paliwa: ";
        cin>>c;
        koszt vw(z,c);          //konstruktor parametrowy
        cout<<"Koszt: "<<vw.wylicz()<<" zl. za km"<<endl;

        return 0;
}
```

Utworzyliśmy dwa obiekty. Pierwszy to opel a drugi to vw. Zobacz że w linijce „*koszt opel;*” zostaje wywołany konstruktor bezparametrowy, a w deklaracji klasy tam mamy delegowanie konstruktora. Czyli wywołanie konstruktora parametrowego z zerami. W linijce która ma w komentarzu (1) wyświetli nam 0 (wartości domyślne). W linijce gdzie w komentarzu mamy (2) widzimy już wyliczony koszt. W obiekcie vw mamy od razu wpisane dane więc od razu jest koszt wyliczany.

Zadanie 4. Zadanie to ma zaprezentować zbudowanie klasy z zastosowaniem szablonu. Klasa nazywa się „kalkulator” i w sumie dodaje dwie cyfry, odejmuje dwie cyfry. Oprócz tego ma dwa konstruktory oraz destruktor oraz dwie metody do wczytywania cyfr. Dzięki zastosowaniu szablonu wykonamy jedną klasę która jest uniwersalna dla różnych typów zmiennych.

Klasa kalkulator:

```
template<class T>
class kalkulator
{
private:
        T liczba1;
        T liczba2;
public:
        kalkulator(void): liczba1(0), liczba2(0) {}
        kalkulator(T a, T b): liczba1(a), liczba2(b) {}
        ~kalkulator(void) {}
```

```
        void liczba1_set(T a);
        void liczba2_set(T b);
        T dodawanie(void);
        T odejmowanie(void);
        void klawiatura(void);
};
```

Dla ułatwienia przedstawię ciało jednej metody

```
template<class T>
void kalkulator<T>::klawiatura(void)
{
        cout<<"Podaj pierwsza liczba: ";
        cin>>liczba1;
        cout<<"Podaj druga liczba: ";
        cin>>liczba2;
}
```

Dopisz brakujące metody. Następnie w funkcji main sprawdź działanie klasy

```
int main(int argc, char** argv) {

        kalkulator<int> kal(3,4);                    //(1)
        cout<<"suma "<<kal.dodawanie()<<endl;

        cout<<"Podaj liczbe w double"<<endl;
        kalkulator<double> moje;                     //(2)
        moje.klawiatura();
        cout<<"roznica "<<moje.odejmowanie()<<endl;


        return 0;
}
```

W funkcji main mamy dwa obiekty typu kalkulator. Pierwszy obiekt nazywa się „kal” i został zbudowany za pomocą klasy kalkulator. Klasa w tym wypadku działa na zmiennych typu int (linia 1). Widzimy to w tym zapisie „<int>”. Lina 1 wywołuje konstruktor parametrowy.

W linii 2 widzimy obiekt który nazywa się „moje”. Został zbudowany za pomocą klasy kalkulator gdzie w tym wypadku klasa będzie działać na zmiennych typu double. Linia 2 pokazuje wywołanie konstruktora bezargumentowego.

Rysunek pokazuje działanie programu. Po uruchomieniu programu użytkownik podał z klawiatury cyfry „3” i „4”. Sprawdź czy też dostałeś takie same wyniki.

Zadanie 5. Program ma za zadanie zliczać cyfry podane z klawiatury przez użytkownika. Program pobiera cyfry i zlicza sumę tak długo aż użytkownik poda cyfrę zero. Cyfra zero kończy pobieranie cyfr i wyświetla sumę. Program został napisany w funkcji main, natomiast cała logika programu jest zawarta w klasie o nazwie „sumator”. Funkcja main jest bardzo prosta i wygląda w ten sposób:

```
int main(int argc, char** argv) {

        sumator S;
        cout<<"Program sumuje liczby, liczba 0 konczy sumowanie"<<endl;

        do
                S.pobierz();
        while(S.koniec());

        cout<<"Suma wprowadzonych liczb to: "<<S.suma();

        return 0;
}
```

Ciała metod klasy sumator wyglądają w ten sposób:

```
void sumator::pobierz(void)
{
        int temp;
        cout<<"Wpisz liczbe aby ja dodac: ";
        cin>>temp;
        if(temp==0)
                _koniec=0;
        else
                _suma+=temp;
}

int sumator::koniec(void)
{
        return _koniec;
}

int sumator::suma(void)
{
        return _suma;
}
```

Dopisz brakującą część programu czyli uzupełnij klasę sumator:

```
class sumator
{
private:

public:

};
```

Działanie programu pokazane jest na rysunku poniżej. Użytkownik podał z klawiatury cyfry: 2, 3, 1, 0. Widzimy że program poprawnie zachował się ponieważ przy wczytaniu 0 skończył wprowadzanie cyfr a następnie podał wynik: 6.

```
Program sumuje liczby, liczba 0 konczy sumowanie
Wpisz liczbe aby ja dodac: 2
Wpisz liczbe aby ja dodac: 3
Wpisz liczbe aby ja dodac: 1
Wpisz liczbe aby ja dodac: 0
Suma wprowadzonych liczb to: 6
--------------------------------
Process exited after 6.884 seconds with return value 0
Aby kontynuować, naciśnij dowolny klawisz . . .
```

Zadanie 6. Napisz program na podstawie klasy z zadania 5 tak aby pobierał cyfry z klawiatury a następnie liczba 0 kończy wczytywanie z klawiatury (tak samo jak w zadaniu 5). Jednak chcę aby wypisać sumę liczb parzystych oraz sumę liczb nieparzystych.

Zadanie 7. Ułóż program z układanki gdzie tworzymy zmienną dynamiczną typu int i ustawiamy wskaźnik który nazywa się temp, wpisujemy do tej zmiennej liczbę 10, wypisujemy na ekran liczbę przez wskaźnik temp, wypisujemy na ekran adres zmiennej dynamicznej, tworzymy drugi wskaźnik który nazywa się ws, ustawiamy wskaźnik ws na zmienną dynamiczną, wpisujemy cyfrę 20 przez wskaźnik ws, wypisujemy na ekran cyfrę wskaźnikiem ws. Aby wskaźniki unique_ptr działały musi być co najmniej kompilator c++14 lub nowszy.

```
unique_ptr<int> temp (new int);
unique_ptr<int> ws;
cout<<temp.get()<<endl;
cout<<*temp<<endl;
*temp=10;
#include <memory>
ws=move(temp);
```

Zadanie 8. Ułóż program z układanki gdzie tworzymy zmienną dynamiczną typu „test” (tak nazywa się klasa) i ustawiamy wskaźnik który nazywa się „t”, wyświetlamy adres zmiennej dynamicznej, wczytujemy liczbę 5, wyświetlamy na ekran terminala cyfrę ze zmiennej dynamicznej, tworzymy drugi wskaźnik który nazywa się „t2” i ustawiamy go na zmienną dynamiczną, wypisujemy na ekran cyfrę za pomocą wskaźnika „t2”. Aby wskaźniki unique_ptr działały musi być co najmniej kompilator c++14 lub nowszy.

```
t->set_a(5);
unique_ptr<test> t(new test);
unique_ptr<test> t2=move(t);
cout<<t.get()<<endl;
cout<<t2->get_a()<<endl;
```

```
cout<<t->get_a()<<endl;
```

Klasa będzie wyglądała w ten sposób:

```
class test
{
private:
   int a;
public:
   test(){ a=0;}
   void set_a(int x) {a=x;}
   int get_a(void) {return a;}
};
```

Zadanie 9. Program pokazuje podstawowe działania na kontenerze „vector”. Tworzymy wektor z czterema cyframi a następnie dodajemy jedną cyfrę 15 na pozycji drugiej. Następnie wypisujemy na ekran wszystkie cyfry. Twoim zadaniem jest wstawić jeszcze liczbę 25 między cyfry 20 a 30 oraz wstawić liczbę 35 między cyfrę 30 a 40.

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
   // Tworzymy wektor liczb całkowitych
   vector<int> vec;

   // Dodajemy elementy do wektora
   // vec.push_back(10);
   // vec.push_back(20);
   // vec.push_back(30);
   // vec.push_back(40);

   //lub dodajemy jedną linią
   vec= {10, 20, 30, 40};

   //dodajemy jeden element 15 na drugim miejscu
   vec.insert(vec.begin() + 1, 15);

   // Wypisujemy elementy wektora
   cout << "Elementy wektora: " << endl;
   for (int i = 0; i < vec.size(); i++) {
      cout << vec[i] << endl;
   }

   return 0;
}
```

Ćwiczenia:

Wykonując zadania w C++ zapisuj wszystkie wykonane programy do pliku tekstowego. Na zakończenie lekcji plik należy skopiować w podane miejsce przez nauczyciela.